

R Quick Reference

STAT 109 — open-notes reference (Modules 0–3 descriptive + inferential patterns)

R Quick Reference

Module 0 — R basics

Task	Code pattern
Assign / print	<code>x <- 10</code> or <code>x = 10</code> , then <code>x</code> by itself. Note that <code>x</code> is a user chosen name.
Vector	<code>v <- c(1, 2, 3, 4)</code> a collection of objects: <code>c</code> is the R function, <code>v</code> is a user chosen name.
Index	<code>v[2]</code> (the element of <code>v</code> in the second position: R counts from 1)
Compare	<code>babies == mothers</code> returns TRUE/FALSE vector for each comparison of objects in the two sets of collections
Count matches	<code>sum(babies == mothers)</code> uses the trick that TRUE = 1 and FALSE = 0
One random shuffle	<code>sample(c(1, 2, 3, 4))</code>
Reproducible Randomness	<code>set.seed(109)</code>
Pre-allocate storage	<code>num_correct <- numeric(10000)</code>
Loop simulation	<code>for (i in 1:10000) { ... num_correct[i] <- ... }</code>
Estimate probability from the results of a for loop simulation	<code>mean(num_correct == 4)</code>
Counts of how many times each object appears	<code>table(num_correct)</code>

```
# Module 0 - baby-matching simulation (pattern)
set.seed(109)
mothers <- c(1, 2, 3, 4)
num_correct <- numeric(10000)
for (i in 1:10000) {
  babies <- sample(c(1, 2, 3, 4))
  num_correct[i] <- sum(babies == mothers)
}
mean(num_correct == 4) # P(all 4 correct)
mean(num_correct == 0) # P(none correct)
```

Module 1 — Data structures & simulation

Task	Code pattern
Type check	<code>class(x)</code>
Integer	<code>y <- 3L</code>
Length	<code>length(v)</code>
Subset range	<code>v[2:4]</code> ; drop 3rd: <code>v[-3]</code>

Task	Code pattern
Logical filter	<code>v[v > 5]</code>
Matrix	<code>m <- matrix(1:6, nrow = 2, ncol = 3)</code>
Matrix size	<code>dim(m)</code>
Matrix index	<code>m[2, 1]; row 1: m[1,]; col 2: m[, 2]</code>
Membership	<code>rolls %in% c(2, 4, 6)</code>
Even die simulation	<code>mean(sample(1:6, 10000, replace = TRUE) %% 2 == 0)</code>

```
v <- c(2, 4, 6, 8, 10)
v[3]
v[v > 5]
m <- matrix(1:6, nrow = 2, ncol = 3)
m[2, 1]
```

Module 1 — Conditional simulation

Task	Code pattern
Build labeled pool	<code>balls <- rep(c("R", "W"), c(3, 3))</code>
Draw without replacement	<code>sample(balls, size = 2, replace = FALSE)</code>
Pre-allocate	<code>successes <- vector("numeric", 10000)</code>
Record with if	<code>if (condition) { successes[k] <- 1 }</code>
AND / OR in if	<code>if (a & b); if (a b)</code>
Which replicates	<code>which(successes == 1)</code>
Weighted sample	<code>sample(c("yes", "no"), n, replace = TRUE, prob = c(0.7, 0.3))</code>
Joint probability	<code>mean(disease == 1 & test == 1)</code>
Conditional from sim	<code>mean(disease == 1 & test == 1) / mean(test == 1)</code>

```
balls <- rep(c("R", "W"), c(3, 3))
replicates <- 10000
successes <- vector("numeric", replicates)
set.seed(109)
for (k in 1:replicates) {
  draw <- sample(balls, 2, replace = FALSE)
  if (draw[1] == "W" & draw[2] == "R") successes[k] <- 1
}
mean(successes)

# Joint event
mean(child == 1 & gummy == 1)
```

Module 1 — Binomial & Project 1

Task	Code pattern
Combinations	<code>choose(n, k)</code>
$P(X = x)$	<code>dbinom(x, size = n, prob = p)</code>
$P(X \leq q)$	<code>pbinom(q, size = n, prob = p)</code>
$P(X > x)$ upper tail	<code>pbinom(x - 1, size = n, prob = p, lower.tail = FALSE)</code> or <code>1 - pbinom(x - 1, ...)</code>
Simulate binomial count	<code>rbinom(10000, size = n, prob = p)</code>
Load plotting	<code>library(tidyverse)</code>

Null binomial graph (milestone & final report)

```
library(tidyverse)

n <- 25      # your study
p <- 0.5     # from H0
x_obs <- 18  # observed successes after data collection

df <- data.frame(
  x = 0:n,
  prob = dbinom(0:n, size = n, prob = p)
)

ggplot(df, aes(x = x, y = prob)) +
  geom_col() +
  geom_vline(xintercept = x_obs, linewidth = 1) +
  labs(x = "x (successes)", y = "P(X = x) under H0")
```

p-value patterns (match H_a direction)

```
# Right-tailed  $H_a$ :  $p > p_0$  - at least  $x_{\text{obs}}$  successes
1 - pbinom(x_obs - 1, size = n, prob = p)

# Left-tailed  $H_a$ :  $p < p_0$ 
pbinom(x_obs, size = n, prob = p)

# Two-tailed (symmetric null) - tail at least as extreme as  $x_{\text{obs}}$ 
# (use smaller of upper/lower doubled, or shade both tails on plot)
```

Descriptive chart for your data

```
# Bar chart of successes vs failures (example)
counts <- c(Success = x_obs, Failure = n - x_obs)
barplot(counts, main = "My sample", ylab = "Count")
```

Module 1 — Quick function index

Function	Role
<code><-</code> or <code>=</code>	assign a value to a name (use <code><-</code> in course code; <code>==</code> compares)
<code>c()</code>	combine into a vector
<code>class()</code>	data type
<code>length()</code> , <code>dim()</code>	size of vector / matrix
<code>matrix()</code>	2D numeric table
<code>sum()</code> , <code>mean()</code>	count TRUE; proportion TRUE
<code>sample()</code>	random draw; use <code>replace =</code> , <code>prob =</code>
<code>rep()</code>	repeat values to build a pool
<code>set.seed()</code>	reproducible randomness
<code>for</code>	repeat simulation
<code>if</code>	branch on condition
<code>which()</code>	indices where TRUE
<code>%in%</code>	element in set?
<code>&</code> , <code> </code>	and / or (element-wise)
<code>choose()</code>	combinations
<code>dbinom()</code> , <code>pbinom()</code> , <code>rbinom()</code>	binomial pmf, cdf, simulate
<code>ggplot</code> , <code>aes</code> , <code>geom_col</code> , <code>geom_vline</code>	null distribution plot

Module 3 — Descriptive EDA

Organized by number and type of variable — matches *Read 0* and *demo lab*. Project 3 is **descriptive only**.

Setup and load

```
library(tidyverse)
dat <- read_csv("https://example.com/data.csv", show_col_types = FALSE)
glimpse(dat)
```

One categorical — counts, proportions, bar chart

Task	Code pattern
Counts per level	<code>dat > count(category)</code>
Add proportions	<code>dat > count(category) > mutate(prop = n / sum(n))</code>
Bar chart	<code>ggplot(dat, aes(x = category)) + geom_bar()</code>
Bar from counts	<code>dat > count(category) > ggplot(aes(x = category, y = n)) + geom_col()</code>

Notation: x = count in one level, n = total,
 $hat{p} = x/n$.

One numerical — center, spread, histogram, boxplot

Task	Code pattern
Summaries	<code>mean(x, na.rm = TRUE), median(x, na.rm = TRUE), sd(x, na.rm = TRUE), IQR(x, na.rm = TRUE)</code>
Histogram	<code>ggplot(dat, aes(x = y)) + geom_histogram(binwidth = ...)</code>
Dot plot	<code>ggplot(dat, aes(x = y)) + geom_dotplot(binwidth = ...)</code>
Boxplot (one variable)	<code>ggplot(dat, aes(y = y)) + geom_boxplot()</code>

Describe: **center**, **spread**, **shape**, **unusual observations**.

Numerical $\times$ categorical — group summaries and boxplots

Task	Code pattern
Summaries by group	<code>dat > group_by(group) > summarize(n = n(), med = median(y, na.rm = TRUE), iqr = IQR(y, na.rm = TRUE))</code>
Side-by-side boxplots	<code>ggplot(dat, aes(x = group, y = y)) + geom_boxplot()</code>
Drop NA before plot	<code>dat > filter(!is.na(y))</code> or <code>drop_na(y)</code>

Optional inferential: `t.test(y ~ group, var.equal = FALSE)` — not required for Project 3.

Two categorical — two-way table and stacked bars

Task	Code pattern
Count table	<code>dat > count(var_a, var_b)</code>
Wide table	<code>... > pivot_wider(names_from = var_b, values_from = n, values_fill = 0)</code>
Row proportions	<code>dat > count(var_a, var_b) > group_by(var_a) > mutate(prop = n / sum(n))</code>
Column proportions	<code>... > group_by(var_b) > mutate(prop = n / sum(n))</code>
Stacked counts	<code>ggplot(dat, aes(x = var_a, fill = var_b)) + geom_bar()</code>
100% stacked	<code>ggplot(dat, aes(x = var_a, fill = var_b)) + geom_bar(position = "fill")</code>

Two numerical — scatterplot and Pearson r

Task	Code pattern
Drop NA	<code>dat > drop_na(x, y)</code>
Scatterplot	<code>ggplot(dat, aes(x = x, y = y)) + geom_point()</code>
Pearson r	<code>cor(x, y)</code> — use only when scatterplot form is roughly linear
Color by group	<code>aes(..., color = group)</code>

Describe: **form** (linear/curved/none), **direction** (positive/negative/none), **strength** (strong/moderate/weak/none). $|r|$ near 1 = strong linear trend; r **misleading** when form is curved.

Analysis pipeline

Step	Code idea	Quick check
1	<code>library(tidyverse)</code>	Packages load without errors
2	<code>read_csv(...); glimpse(dat)</code>	Column names and types match Part 1
3	<code>mutate(... factor(..., levels = ...))</code>	Levels match your two groups / outcome categories
4	<code>group_by</code> + summarise and/or count	Means/SDs if numerical; counts if categorical
5	<code>ggplot</code>	Boxplot/histogram for numerical by group; bar chart for categorical
6	Test + CI	<code>t.test(..., var.equal = FALSE)</code> or <code>prop.test</code> / <code>chisq.test</code>
7	Short interpretation	<i>p</i> -value addresses chance only

Setup

```
if (!requireNamespace("tidyverse", quietly = TRUE)) {
  install.packages("tidyverse", repos = "https://cloud.r-project.org")
}
library(tidyverse)
```

Import and inspect

Task	Code pattern
Load CSV	<code>dat <- read_csv("file.csv", show_col_types = FALSE)</code>
Peek structure	<code>glimpse(dat)</code>
First rows	<code>head(dat, 10)</code>

Categorical variables (factors)

Task	Code pattern
Set levels / order	<code>dat <- dat > mutate(group = factor(group, levels = c("A", "B")))</code>
Check levels	<code>levels(dat\$group)</code>

Summaries

Setting	Code pattern
One numerical variable	<code>dat > summarize(mean_y = mean(y), sd_y = sd(y), n_y = n())</code>

Setting	Code pattern
Numerical by two groups	<code>dat > group_by(group) > summarize(n = n(), mean_y = mean(y), sd_y = sd(y))</code>
Counts for one categorical 2 × 2 counts	<code>dat > count(category) > mutate(prop = n / sum(n))</code> <code>dat > count(group, outcome) then pivot_wider</code>

Plots (ggplot2)

Setting	Pattern
Numerical response, two groups	<code>ggplot(dat, aes(x = group, y = y)) + geom_boxplot()</code>
Two categorical	<code>dat > count(group, outcome) > ggplot(aes(x = group, y = n, fill = outcome)) + geom_col()</code>
Two numerical	<code>ggplot(dat, aes(x = x1, y = x2)) + geom_point()</code>
Subclassification	Add <code>facet_wrap(~ confounder)</code>

Always add `labs(title = ..., x = ..., y = ...)` with **units** where relevant.

Two-sample tests and confidence intervals

Response type	R command	Notes
Numerical , 2 groups	<code>t.test(y ~ group, data = dat, var.equal = FALSE)</code>	Welch <i>t</i> -test; includes 95% CI
Binary , 2 groups	<code>prop.test(x = c(x1, x2), n = c(n1, n2), correct = FALSE)</code>	Compares two proportions
Two categorical variables	<code>chisq.test(tab)</code>	Check <code>chisq.test(tab)\$expected</code> for 5 rule

Simulating small datasets

Task	Example pattern
Build tibble	<code>tibble(group = rep(c("A", "B"), each = 30), y = c(rnorm(30, 10, 2), rnorm(30, 12, 2)))</code>
Binary outcome	<code>rbinom(n, 1, prob = ...) inside mutate or c(...)</code>
Save	<code>write_csv(dat, "project4_data.csv")</code>